BIAS, VARIANCE AND GENERALISATION PROBLEM SHEET

This sheet covers the lectures on bias and variance, over-fitting and symmetry. Question 1 practises the basic ideas, Question 2 works through shrinkage in detail and Question 3 ridge regression. Each ends with a short Python part (`import numpy as np` is assumed). Question 4 is harder and optional: it shows that gradient descent regularises, and builds an invariant function by averaging over a group.

1

(a) The same machine is trained on four different data sets. At one input x, where the true output is $y = 2.5$, the four machines predict 2.0, 2.4, 1.6 and 2.0. Find the mean machine's prediction, the bias $(\hat{f}_m - y)^2$ and the variance at this input, and check that they add up to the average squared error.

$\hat{f}_m = (2.0 + 2.4 + 1.6 + 2.0)/4 = 2.0$, so the bias is $(2.0 - 2.5)^2 = 0.25$. The variance is $\frac{1}{4}(0^2 + 0.4^2 + 0.4^2 + 0^2) = 0.08$. The average squared error is $\frac{1}{4}(0.25 + 0.01 + 0.81 + 0.25) = 0.33 = 0.25 + 0.08$.

[4 marks]

(b) For each pair, which machine would you expect to have the higher bias, and which the higher variance: (i) a straight line and a degree-15 polynomial fitted to 20 noisy points; (ii) k-nearest neighbours with $k = 1$ and with $k = 50$? Explain briefly.

(i) The straight line has the higher bias (it cannot follow a curved function); the degree-15 polynomial has the higher variance (it can pass close to every noisy point, so it changes a lot from data set to data set). (ii) $k = 1$ copies the label of the single nearest point, so it has low bias and high variance; $k = 50$ averages over many points, so it has low variance but higher bias (it smooths over real detail).

[4 marks]

(c) You have 1000 labelled examples and use 5-fold cross validation. How many models do you train, how many examples does each see, and how many examples is each validated on? Why must you not measure performance on the training data?

Five models, each trained on 800 examples and validated on the remaining 200; every example is used for validation exactly once. Training error is optimistic: a flexible machine can fit spurious rules in the training data that do not hold on unseen data.

[3 marks]

(d) An ensemble averages M predictors, each with variance $s^2 = 1$ at some input and pairwise correlation $\rho = 0.3$. Using $\mathrm{Var}(\bar{f}_M) = \rho s^2 + (1 - \rho)s^2/M$, find the variance for $M = 10$ and as $M \to \infty$. Does averaging change the bias?

For $M = 10$: $0.3 + 0.7/10 = 0.37$. As $M \to \infty$ it falls to the floor $\rho s^2 = 0.3$. Averaging does not change the mean, so the bias is unchanged.

[3 marks]

(e) In Python, take 20 equally spaced inputs on $[0,1]$ and targets $\sin(2\pi x) + 0.3\epsilon$ with $\epsilon \sim \mathcal{N}(0,1)$. For polynomials of degree 1, 3 and 9, fit 1000 independent data sets (`np.polyfit`) and estimate the bias (squared error of the mean fit, averaged over 100 test points) and the variance.

```
rng = np.random.default_rng(0)
x = np.linspace(0, 1, 20)
xt = np.linspace(0, 1, 100)
f = lambda x: np.sin(2 * np.pi * x)
for d in [1, 3, 9]:
    P = []
    for _ in range(1000):
        y = f(x) + 0.3 * rng.standard_normal(20)
        P.append(np.polyval(np.polyfit(x, y, d), xt))
    P = np.array(P)
    bias2 = ((P.mean(0) - f(xt))**2).mean()
    print(d, bias2, P.var(0).mean())
```

Bias and variance are about 0.207 and 0.009 for degree 1, 0.005 and 0.017 for degree 3, and 0.000 and 0.043 for degree 9. The bias falls and the variance rises with the degree; degree 3 has the smallest total (0.022).

[6 marks]

End of question 1

Answers

2 We estimate the mean θ of a distribution with variance σ^2 from m independent samples, whose sample mean is $\bar{x}$. Instead of $\bar{x}$ we use the *shrinkage* estimate $\hat{\theta} = c\bar{x}$ with $0 \leq c \leq 1$.

(a) Find $\mathbb{E}\left[\hat{\theta}\right]$ and $\mathrm{Var}(\hat{\theta})$. For which c is $\hat{\theta}$ unbiased?

$\mathbb{E}\left[\bar{x}\right] = \theta$ and $\mathrm{Var}(\bar{x}) = \sigma^2/m$, so $\mathbb{E}\left[\hat{\theta}\right] = c\theta$ and $\mathrm{Var}(\hat{\theta}) = c^2\sigma^2/m$. It is unbiased only for $c = 1$ (or if $\theta = 0$).

[4 marks]

(b) Show that the mean squared error is $\mathbb{E}\left[(\hat{\theta} - \theta)^2\right] = (1-c)^2\theta^2 + c^2\sigma^2/m$, and identify the bias and variance terms.

Adding and subtracting $\mathbb{E}\left[\hat{\theta}\right] = c\theta$,

$$\mathbb{E}\left[(\hat{\theta} - \theta)^2\right] = \mathbb{E}\left[(\hat{\theta} - c\theta)^2\right] + (c\theta - \theta)^2 = c^2\frac{\sigma^2}{m} + (1-c)^2\theta^2,$$

since the cross term $2(c\theta - \theta)\mathbb{E}\left[\hat{\theta} - c\theta\right]$ is zero. $(1-c)^2\theta^2$ is the bias and $c^2\sigma^2/m$ the variance.

[4 marks]

(c) Find the c^* that minimises the mean squared error and the minimum error. Show that it is smaller than σ^2/m, the error of $\bar{x}$, whenever $\theta \neq 0$.

Setting the derivative $-2(1-c)\theta^2 + 2c\sigma^2/m$ to zero,

$$c^* = \frac{\theta^2}{\theta^2 + \sigma^2/m}, \qquad \text{error} = \frac{\sigma^2}{m}\frac{\theta^2}{\theta^2 + \sigma^2/m} < \frac{\sigma^2}{m}.$$

(Substituting c^*, the bias is $\theta^2(\sigma^2/m)^2/(\theta^2 + \sigma^2/m)^2$ and the variance $(\sigma^2/m)\theta^4/(\theta^2 + \sigma^2/m)^2$; they add to the expression above.) The fraction is less than 1, so a biased estimate beats the unbiased one.

[5 marks]

(d) Why can we not use c^* directly? What happens to c^* as $m \to \infty$, and why does that make sense?

c^* depends on the unknown θ we are trying to estimate; in practice the amount of shrinkage is chosen by validation (or from a prior). As $m \to \infty$, $\sigma^2/m \to 0$ and $c^* \to 1$: with plenty of data the variance is already small, so there is nothing to gain from adding bias.

[3 marks]

Answers

(e) Take $\theta = 2$, $\sigma = 4$, $m = 10$. Compute c^* and the errors for $c = 1$ and $c = c^*$. Check by simulation in Python, drawing 100000 samples of size m.

$\sigma^2/m = 1.6$, so $c^* = 4/5.6 \approx 0.714$, with error $1.6 \times 4/5.6 \approx 1.143$ against 1.6 for $c = 1$.

```
rng = np.random.default_rng(0)
theta, sigma, m = 2.0, 4.0, 10
xbar = rng.normal(theta, sigma, (100000, m)).mean(1)
for c in [1.0, 0.714, 0.5]:
    print(c, np.mean((c * xbar - theta)**2))
```

This gives 1.596, 1.137 and 1.395 (theory 1.6, 1.143 and 1.4): shrinking too far ($c = 0.5$) helps less, but still beats $c = 1$.

[4 marks]

Answers

End of question 2

3 Data are generated as $\boldsymbol{y} = \mathbf{X}\boldsymbol{w}^* + \boldsymbol{\epsilon}$, where the noise terms ϵ_k are independent with mean 0 and variance σ^2. Ridge regression minimises $\|\mathbf{X}\boldsymbol{w} - \boldsymbol{y}\|^2 + \eta\|\boldsymbol{w}\|^2$.

(a) In one dimension, $y_k = w^* x_k + \epsilon_k$ for $k = 1,\ldots,m$. With $S = \sum_k x_k^2$, show that $\hat{w} = \sum_k x_k y_k/(S+\eta)$ and find its mean and variance. For $x = (1,2,3)$, $w^* = 0.5$ and $\sigma = 1$, find the bias, variance and mean squared error for $\eta = 0$ and $\eta = 4$.

The derivative $2\sum_k x_k(wx_k - y_k) + 2\eta w$ is zero at $\hat{w} = \sum_k x_k y_k/(S+\eta)$. Since $\sum_k x_k y_k = Sw^* + \sum_k x_k \epsilon_k$,

$$\mathbb{E}[\hat{w}] = \frac{S}{S+\eta} w^*, \qquad \mathrm{Var}(\hat{w}) = \frac{\sigma^2 S}{(S+\eta)^2}.$$

Here $S = 14$. For $\eta = 0$: no bias, variance $1/14 \approx 0.071$. For $\eta = 4$: bias $(-4 \times 0.5/18)^2 \approx 0.012$, variance $14/324 \approx 0.043$, total 0.056, which is smaller.

[5 marks]

(b) In general write the SVD $\mathbf{X} = \mathbf{U}\mathbf{S}\mathbf{V}^\mathsf{T}$, with columns $\boldsymbol{u}_k$ and $\boldsymbol{v}_k$ and singular values s_k. Show that $\mathbf{X}^\mathsf{T}\mathbf{X} + \eta\mathbf{I} = \mathbf{V}(\mathbf{S}^2 + \eta\mathbf{I})\mathbf{V}^\mathsf{T}$, and hence that the component of $\hat{\boldsymbol{w}}$ along $\boldsymbol{v}_k$ is $\hat{w}_k = s_k \boldsymbol{u}_k^\mathsf{T}\boldsymbol{y}/(s_k^2 + \eta)$.

$\mathbf{X}^\mathsf{T}\mathbf{X} = \mathbf{V}\mathbf{S}\mathbf{U}^\mathsf{T}\mathbf{U}\mathbf{S}\mathbf{V}^\mathsf{T} = \mathbf{V}\mathbf{S}^2\mathbf{V}^\mathsf{T}$ and $\mathbf{I} = \mathbf{V}\mathbf{V}^\mathsf{T}$ ($\mathbf{V}$ is orthogonal), which gives the first result. Its inverse is $\mathbf{V}(\mathbf{S}^2 + \eta\mathbf{I})^{-1}\mathbf{V}^\mathsf{T}$, so

$$\hat{\boldsymbol{w}} = \mathbf{V}(\mathbf{S}^2 + \eta\mathbf{I})^{-1}\mathbf{V}^\mathsf{T}\mathbf{V}\mathbf{S}\mathbf{U}^\mathsf{T}\boldsymbol{y} = \mathbf{V}(\mathbf{S}^2 + \eta\mathbf{I})^{-1}\mathbf{S}\mathbf{U}^\mathsf{T}\boldsymbol{y}.$$

Multiplying by $\boldsymbol{v}_k^\mathsf{T}$ picks out row k of the diagonal matrix: $\hat{w}_k = s_k \boldsymbol{u}_k^\mathsf{T}\boldsymbol{y}/(s_k^2 + \eta)$.

[5 marks]

(c) With $w_k^* = \boldsymbol{v}_k^\mathsf{T}\boldsymbol{w}^*$, show that $\boldsymbol{u}_k^\mathsf{T}\boldsymbol{y} = s_k w_k^* + \boldsymbol{u}_k^\mathsf{T}\boldsymbol{\epsilon}$, and hence that $\mathbb{E}[\hat{w}_k] = s_k^2 w_k^*/(s_k^2 + \eta)$ and $\mathrm{Var}(\hat{w}_k) = \sigma^2 s_k^2/(s_k^2 + \eta)^2$.

$\boldsymbol{u}_k^\mathsf{T}\mathbf{X}\boldsymbol{w}^* = \sum_j s_j(\boldsymbol{u}_k^\mathsf{T}\boldsymbol{u}_j)(\boldsymbol{v}_j^\mathsf{T}\boldsymbol{w}^*) = s_k w_k^*$, because the $\boldsymbol{u}_j$ are orthonormal. The noise $\boldsymbol{u}_k^\mathsf{T}\boldsymbol{\epsilon} = \sum_i u_{ik}\epsilon_i$ has mean 0 and variance $\sigma^2\sum_i u_{ik}^2 = \sigma^2$. Substituting into part (b) gives both results.

[4 marks]

Answers

(d) Take two components with $s = (2, 0.1)$, $w^* = (1, 1)$ (in the $\boldsymbol{v}$ basis) and $\sigma = 1$, so that $\mathbb{E}\left[\|\hat{\boldsymbol{w}} - \boldsymbol{w}^*\|^2\right] = \sum_k \frac{\eta^2 w_k^{*2} + \sigma^2 s_k^2}{(s_k^2 + \eta)^2}$. Evaluate it for $\eta = 0$ and $\eta = 1$. Show that each term is smallest at $\eta = \sigma^2 / w_k^{*2}$, and check your values by simulation in Python.

$\eta = 0$: $1/4 + 1/0.01 = 100.25$, almost all from the small singular value. $\eta = 1$: $\frac{1}{25} + \frac{1}{1.01^2} \approx 0.2 + 0.99 = 1.19$. Differentiating a term $(\eta^2 a + b)/(s^2 + \eta)^2$ (with $a = w_k^{*2}$, $b = \sigma^2 s^2$) gives a numerator proportional to $\eta a s^2 - b = s^2(\eta a - \sigma^2)$, which is zero at $\eta = \sigma^2 / a$, whatever s is. Here every term is smallest at $\eta = 1$.

```
rng = np.random.default_rng(0)
U, _ = np.linalg.qr(rng.standard_normal((20, 2)))
V, _ = np.linalg.qr(rng.standard_normal((2, 2)))
s = np.array([2.0, 0.1])
X = U @ np.diag(s) @ V.T
w_star = V @ np.array([1.0, 1.0])
for eta in [0.0, 0.1, 1.0]:
    err = []
    for _ in range(20000):
        y = X @ w_star + rng.standard_normal(20)
        A = X.T @ X + eta * np.eye(2)
        w = np.linalg.solve(A, X.T @ y)
        err.append(((w - w_star)**2).sum())
    print(eta, np.mean(err))
```

The simulation gives 99.8, 1.88 and 1.19, against 100.25, 1.89 and 1.19 from the formula.

[6 marks]

End of question 3

Answers

4 This question is harder and optional.

(a) We minimise $\frac{1}{2}\|\mathbf{X}\boldsymbol{w} - \boldsymbol{y}\|^2$ by gradient descent, $\boldsymbol{w} \leftarrow \boldsymbol{w} - r\mathbf{X}^\mathsf{T}(\mathbf{X}\boldsymbol{w} - \boldsymbol{y})$, starting from $\boldsymbol{w} = \mathbf{0}$, where $\mathbf{X}$ is $m \times p$ with $p > m$ and $\mathbf{X}\mathbf{X}^\mathsf{T}$ invertible. Explain why $\boldsymbol{w}$ is always of the form $\mathbf{X}^\mathsf{T}\boldsymbol{a}$. If it converges to an exact fit, show that the limit is $\boldsymbol{w} = \mathbf{X}^\mathsf{T}(\mathbf{X}\mathbf{X}^\mathsf{T})^{-1}\boldsymbol{y}$.

Each step adds $-r\mathbf{X}^\mathsf{T}(\mathbf{X}\boldsymbol{w} - \boldsymbol{y})$, which has the form $\mathbf{X}^\mathsf{T}\boldsymbol{b}$; starting from $\mathbf{0}$, a sum of such vectors is still $\mathbf{X}^\mathsf{T}\boldsymbol{a}$. An exact fit with $\boldsymbol{w} = \mathbf{X}^\mathsf{T}\boldsymbol{a}$ needs $\mathbf{X}\mathbf{X}^\mathsf{T}\boldsymbol{a} = \boldsymbol{y}$, so $\boldsymbol{a} = (\mathbf{X}\mathbf{X}^\mathsf{T})^{-1}\boldsymbol{y}$.

[6 marks]

(b) Show that this is the exact fit with the smallest norm. (Hint: any other exact fit is $\boldsymbol{w} + \boldsymbol{z}$ with $\mathbf{X}\boldsymbol{z} = \mathbf{0}$.) Why does this matter for over-parameterised models such as neural networks?

If $\mathbf{X}\boldsymbol{z} = \mathbf{0}$ then $\boldsymbol{z}^\mathsf{T}\boldsymbol{w} = \boldsymbol{z}^\mathsf{T}\mathbf{X}^\mathsf{T}\boldsymbol{a} = (\mathbf{X}\boldsymbol{z})^\mathsf{T}\boldsymbol{a} = 0$, so by Pythagoras $\|\boldsymbol{w} + \boldsymbol{z}\|^2 = \|\boldsymbol{w}\|^2 + \|\boldsymbol{z}\|^2 \geq \|\boldsymbol{w}\|^2$. With more parameters than data there are infinitely many exact fits; gradient descent from a small start picks a small-norm one without being told to. The optimiser is a regulariser, which helps explain why heavily over-parameterised models can still generalise (double descent).

[6 marks]

(c) Check part (a) in Python with $m = 10$, $p = 30$, random $\mathbf{X}$ and $\boldsymbol{y}$, learning rate $r = 0.01$ and 20000 steps.

```
rng = np.random.default_rng(0)
m, p = 10, 30
X = rng.standard_normal((m, p))
y = rng.standard_normal(m)
w = np.zeros(p)
for _ in range(20000):
    w -= 0.01 * X.T @ (X @ w - y)
w_mn = X.T @ np.linalg.solve(X @ X.T, y)
print(np.abs(w - w_mn).max(), np.abs(X @ w - y).max())
```

Both differences are about 10^{-15}: gradient descent has found the minimum-norm fit. ($r = 0.01$ is below $2/s_{\max}^2 \approx 0.024$, so the iteration is stable.)

[5 marks]

Answers

(d) The rotations of a square image by 0, 90, 180 and 270 degrees form a group C_4. For any function f of images, show that $\bar{f}(\boldsymbol{x}) = \frac{1}{4}\sum_{k=0}^{3} f(R^k \cdot \boldsymbol{x})$, where R is a rotation by 90 degrees, is unchanged when $\boldsymbol{x}$ is rotated. Check this in Python with `np.rot90` for $f(\boldsymbol{x}) = \sum_{ij} x_{ij}(4i + j)$ on a random 4×4 image.

$\bar{f}(R \cdot \boldsymbol{x}) = \frac{1}{4}\sum_{k=0}^{3} f(R^{k+1} \cdot \boldsymbol{x})$. As k runs over $0,\ldots,3$, $k+1$ runs over $1,\ldots,4$, and $R^4 = e$, so this is the same four terms in a different order: $\bar{f}(R \cdot \boldsymbol{x}) = \bar{f}(\boldsymbol{x})$.

```
f = lambda x: (x * np.arange(16).reshape(4, 4)).sum()
def fbar(x):
    return sum(f(np.rot90(x, k)) for k in range(4)) / 4
img = rng.standard_normal((4, 4))
print([f(np.rot90(img, k)) for k in range(4)])
print([fbar(np.rot90(img, k)) for k in range(4)])
```

f takes four different values on the four rotations, while $\bar{f}$ takes the same value on all of them. This is test-time augmentation: averaging over a group makes any model invariant.

[8 marks]

Answers

End of question 4

END OF PAPER